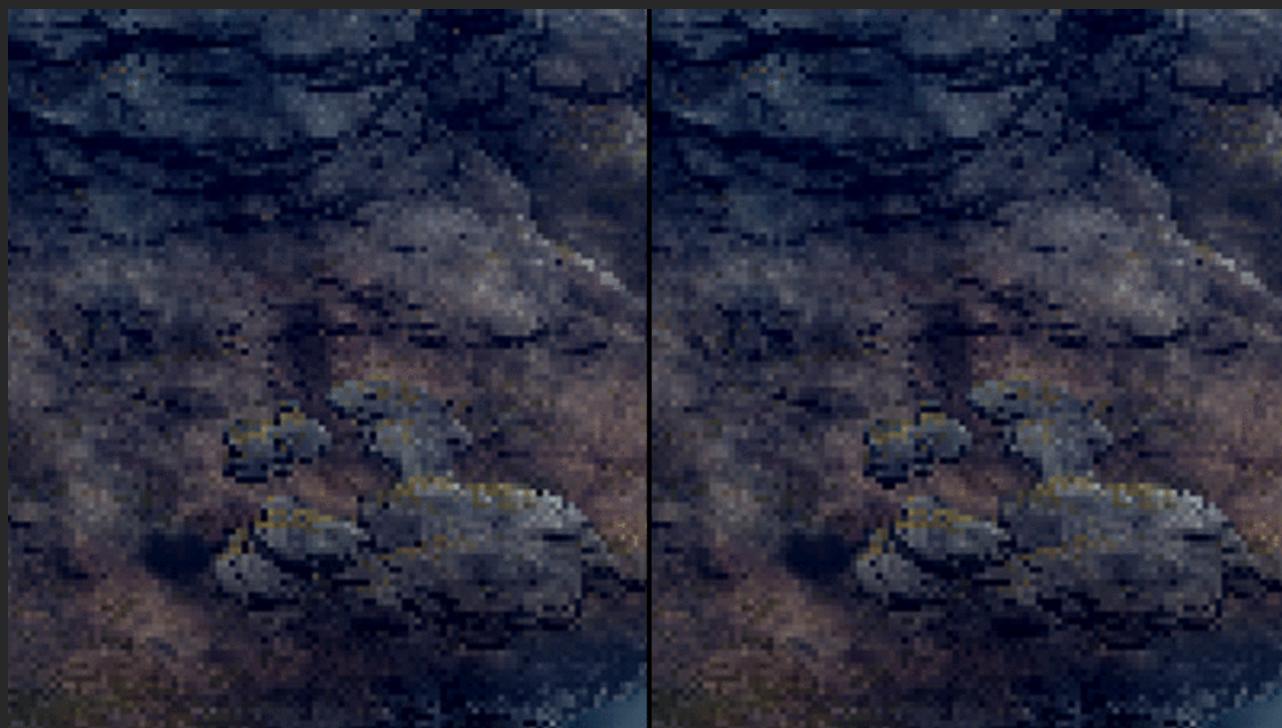


Inigo Quilez :: articles :: don't flip, reflect and clip - 2014



Intro

So, there's this simple problem in computer graphics that I've encountered multiple times: a vector that I've generated through some computation is facing the wrong side of the hemisphere that I need it to be in. The naive way to fix it is to simply flip or negate it so it falls in the correct hemisphere, but that can create all sorts of problems.



Left: flipping normals introduces wrong colors (note brown pixels in dark areas).

Right: clipping normals keeps the image stable

Let me give you two examples where the fix fails:

Example 1: say we are dynamically generating procedural grass on the surface of a rock. So, for many points

on its surface we generate blades with random length, thickness, stiffness, color and orientation. The orientation of each blade needs to be in the outer hemisphere around the rock's surface normal, so we generate a random point in space and if it's in the wrong side of the tangent plane we flip it. So far so good, that's easy and the render looks great. But when rendering the next frame because of the renderer's precision limitations the rock's surface normal changes a tiny little bit. This happens more often than you think with subdivision surfaces for example due to their view dependent refinement, or with procedural meshes that are themselves the result of computations, or with SDF objects, or even with plain static meshes when their transforms have been baked. Whatever the reason, even though the normals look stable to our naked eye, their numerical value can slightly change frame to frame. And with it, the hemispheres of valid directions for our grass blades change too. This means that some number of the blades of grass that got generated in the correct hemisphere in the previous frame now fall in the wrong hemisphere in the current frame, and need a flip. Suddenly we have an animation with flicker and popping errors. Now, while this is infrequent in a shot with millions of blades of grass it is guaranteed it's going to happen to a few blades... and we only need a single one them to be visible to the camera to ruin the whole shot.

Example 2: imagine we are rendering an SDF through raymarching in GLSL, and we have computed normals for it through finite differences. We are now performing lighting but we notice that every now and then the lighting blows up and gets a white pixel in the image, especially around object silhouettes. We realize it's due to normals facing away from the ray direction, so we force them to look towards the camera with `faceforward()`, which essentially flips the normal. The white pixels are gone, but their color is still not right, because we've effectively turned the surface inside out, so we are getting totally picking the wrong lighting. And we do not like wrong pixels if we can have the correct pixels.

The problem

So, the problem in that vector flipping or negation is no a continuous map. If we look at the naive solution

```
// flip v if it's in the negative half plane defined by r
vec flipVec( vec v, vec r )
{
    float k = dot(v,r);
    return (k>0.0) ? v : -v;
}
```

we see that even the tiniest change in `v` or `r` can send `v` all the way in the opposite direction. So, what we need to do is find a method to bring `v` to the correct hemisphere through a function that is continuous rather

than discontinuous. Luckily there are two easy ways to do it:

Solution

The simplest solution is to reflect the vector $\mathbf{v}$ along the plane defined by $\mathbf{r}$. This works and produces a continuous transformation because a vector that is very close to the plane separating the two hemispheres will reflect to the other side into a location that is also very close to the plane. In other words, small changes in the input always produce small changes in the output, which is what brings stability to this approach. This is the code that implements it (I left the dimensionality of the vector types unspecified since this works in any number of dimensions):

```
// reflect v if it's in the negative half plane defined by r
vec reflVec( vec v, vec r )
{
    float k = dot(v,r);
    return (k>0.0) ? v : v-2.0*r*k;
}
```

Alternatively, if for some reason you prefer to snap $\mathbf{v}$ to the half-plane, you can use the following function, although it is a bit slower than the reflection:

```
// clip v if it's in the negative half plane defined by r
vec3 clipVec( vec v, vec r )
{
    float k = dot(v,r);
    return (k>0.0) ? v : (v-r*k)*inversesqrt(1.0-k*k/dot(v,v));
}
```

Note that if $\mathbf{v}$ is normalized, the expression simplifies a little bit and you can remove the division by $\text{dot}(\mathbf{v},\mathbf{v})$. And/or if you only need to catch vectors that penetrate slightly in the negative hemisphere then you can assume $k^2 \approx 0$ and replace $\text{inversesqrt}(1-k^2)$ by its [Taylor approximation](#) $1.0-0.5*k^2$, or even by 1. You can also remove the `inversesqrt()` completely if you plan to normalize $\mathbf{v}$ _after_ the clip rather than before, leaving you with:

```
// clip v if in the negative half plane defined by r. NOTE - doesn't preserve length
vec clipVecNoLength( vec v, vec r )
{
    float k = dot(v,r);
```

```
return (k>0.0) ? v : v-r*k;  
}
```

Conclusion

The following animation is a comparison of the three methods. The yellow arrows are the result of applying the hemisphere fix to the grey arrow. Pay attention to the movement of the yellow arrows, and note how the "Flip" version jumps from one quadrant to the other when the arrow gets into the negative hemisphere. The "Reflect" and "Clip" in the other hand never jump, which is exactly what we want:



You can see both techniques in action and the reference code in the following realtime shader in Shadertoy:

<https://www.shadertoy.com/view/4dBXz3>

inigo quilez - learning computer graphics since 1994